

Cartridge & Cloud - Build & Versioning Guide

Versiones, ramas, builds, tags y changelog desde cero

VRM Games / Blas Luis Rocha González

21/06/2026

1. Esquema de versión
2. Ramas
3. Tags propuestos
4. Tipos de build
5. Nombre
6. Build metadata
7. Checklist previo
8. Checklist posterior
9. Changelog
10. Compatibilidad de saves
11. Steam

Proyecto: Cartridge & Cloud
Título técnico provisional: Cartridge & Cloud
Plataforma inicial: PC / Steam
Motor previsto: Unity 6 LTS / C#
Versión del documento: v0.3
Estado: Preproducción reiniciada desde cero
Fuente conceptual: `Enfoque_v0.6.md`

Regla de interpretación: el proyecto se documenta como una preproducción nueva. No se presupone código, escenas, managers, datos, builds ni sprints implementados. Todo elemento técnico descrito es una especificación o recomendación hasta que exista evidencia de implementación y QA.

1. Esquema de versión

SemVer adaptado:

- `v0.0.x` : preproducción, setup y spikes.
- `v0.1.0` : primer bucle técnico integrado.
- `v0.2.0` : prototipo jugable.
- `v0.3.0` : vertical slice.
- `v0.4+` : MVP y expansiones.
- `v1.0.0` : release comercial.

Los números concretos pueden revisarse en Sprint 0, pero no se reutilizan tags del concepto anterior.

2. Ramas

`main` estable; ramas `feature/`, `fix/`, `docs/`, `release/`. Desarrollo en solitario puede integrar rápidamente, pero cada cambio debe compilar y pasar smoke tests.

3. Tags propuestos

Tag	Hito
v0.0.1-project-foundation	Sprint 0
v0.0.2-bootstrap-scene-flow	Sprint 1

Tag	Hito
v0.1.0-world-construction-prototype	Sprints 3-5
v0.1.1-inventory-logistics	Sprints 6-8
v0.2.0-customer-sales-loop	Sprints 9-13
v0.2.1-save-ui-integration	Sprints 14-15
v0.3.0-vertical-slice	Sprint 17

4. Tipos de build

Editor Test, Dev Build, QA Build, Private Tester, Vertical Slice Candidate, Alpha, Beta, RC, Release.

5. Nombre

```
CAC_v0.3.0_VerticalSlice_Windows_x64_2026-XX-XX_Build001.zip
```

6. Build metadata

Versión, commit SHA, fecha UTC, tipo, Unity version, save version, content version y entorno.

7. Checklist previo

Compilación, tests, validación de contenido, escenas, guardado, core loop, logs, localización, licencias, número de versión y espacio en disco.

8. Checklist posterior

Ejecutar fuera del editor, instalación limpia, smoke test, save/load, logs, hash SHA-256, archivado de reporte y actualización de changelog.

9. Changelog

Categorías Added, Changed, Fixed, Removed, Known Issues. No incluir afirmaciones no verificadas.

10. Compatibilidad de saves

Cada build declara `saveVersion`. Los breaking changes requieren migración o aviso explícito. Las builds de prototipo pueden invalidar saves solo si se documenta antes de distribuirlas.

11. Steam

Los depots y ramas Steam no se crean como parte del setup inicial. Se incorporan cuando el Steam Publishing Plan autorice la fase.